Python Weekly Assignments

Each assignment is designed to help you practice and strengthen your programming skills. Assignments are due **one week** after they are issued. Be sure to test your code thoroughly, comment where necessary, and submit your work on time.

Instructions

1. Write **separate python programs** for each given task.
2. Ensure that your code is properly **indented and commented** to explain key steps.
3. Use **appropriate data types** for each input/output.
4. Save your files with meaningful names (e.g., hospital.py, grading_system.py).
5. Submit source code files before the deadline.

Submission Guidelines

- Submit your .py files through github.
- Ensure the code compiles and runs without errors.

1. TOPIC 1: INTRODUCTION TO PYTHON

Task 1: Hospital Patient System

A hospital wants to develop a simple Python program to record basic patient information. The hospital has chosen Python because of its simplicity and readability. Write a Python program that performs the following tasks:

i. Create variables to store the following patient details:

- Patient name
- Patient ID
- Patient age
- Patient gender

ii. Assign suitable values to the variables.

iii. Display the patient details using the print() function.

iv) Use an **f-string** to display the information in the following format:

```
Patient Name: John Kamau
Patient ID: P001
Age: 35
Gender: Male
```

v) Add comments to the program to explain the major sections of the code.

Task 2: University Python Program

A university wants to introduce Python programming to first-year students. The lecturer wants students to demonstrate the difference between interactive programming and script-based programming. Write a Python program that performs the following tasks:

i. Create a Python script named student.py.

ii. Create variables to store:

- Student name
- Registration number
- Course
- Year of study

iii. Assign suitable values to the variables.

iv) Display all the student information using the print() function.

v) State the difference between **Python interactive mode** and **Python script mode**.

vi. State which mode is more appropriate for developing a complete student management application and give a reason.

Task 3: Python Applications in Business

A small business wants to use Python to automate some of its daily activities. Write a Python program that demonstrates how Python can be used in a business environment. The program should perform the following tasks:

i. Create variables to store:

- Business name
- Product name
- Product price
- Quantity purchased

ii. Assign appropriate values to the variables.

iii. Calculate the total purchase cost using:

Total Cost = Product Price × Quantity

iv) Display the business name, product name, quantity, price and total cost.

v) State **three characteristics of Python** that make it suitable for developing business applications.

2. TOPIC 2: PYTHON BASICS

Task 1: Employee Information

A company wants a simple Python program to record and display employee information. Write a Python program that performs the following tasks:

i. Create variables to store:

- Employee name
- Employee ID
- Employee age
- Basic salary

ii. Accept the employee name, ID, age and basic salary from the keyboard using input().

iii. Convert:

- Age to an integer using int()
- Basic salary to a floating-point number using float()

iv) Calculate the annual salary using:

Annual Salary = Basic Salary × 12

v) Display the employee details using an **f-string**.

vi. Display the salary values to **two decimal places**.

Task 2: Supermarket Purchase

A supermarket wants to calculate the total cost of products purchased by a customer. Write a Python program that performs the following tasks:

i. Ask the customer to enter:

- Customer name

- Product name
- Quantity purchased
- Price per item

ii. Convert the quantity to an integer and the price to a float.

iii. Calculate the total cost using:

Total Cost = Quantity × Price

iv) Display the following information using an f-string:

- Customer name
- Product name
- Quantity
- Price
- Total cost

v) Display the price and total cost to **three decimal places**.

vi. Include a **single-line comment** explaining the calculation.

Task 3: Employee Status

A company wants to record whether an employee is currently active. Write a Python program that performs the following tasks:

i. Create variables for:

- Employee name
- Employee age
- Employee salary
- Employee active status

ii. Assign suitable values to the variables.

iii. Store the employee active status using a **Boolean value** (True or False).

iv) Display all employee information using f-string formatting.

v) Demonstrate how a number can be converted to a string using str() before concatenating it with another string.

vi. Include a **multi-line comment** describing the purpose of the program.

3. TOPIC 3: PYTHON OPERATORS

Task 1: Employee Salary Calculation

A company calculates an employee's net salary by adding a 10% housing allowance to the basic salary and subtracting a fixed deduction. Write a Python program that performs the following tasks:

i. Ask the user to enter:

- Employee name
- Basic salary
- Monthly deduction

ii. Convert the salary and deduction to floating-point numbers.

iii. Calculate the housing allowance using:

Housing Allowance = 10% × Basic Salary

iv) Calculate the gross salary using:

Gross Salary = Basic Salary + Housing Allowance

v) Calculate the net salary using:

Net Salary = Gross Salary − Deduction

vi. Display the basic salary, allowance, gross salary, deduction and net salary.

Task 2: Supermarket Discount

A supermarket gives customers a 15% discount on purchases of KSh 10,000 or more. Write a Python program that performs the following tasks:

i. Ask the customer to enter the total purchase amount.

ii. Convert the amount to a floating-point number.

iii. Use a comparison operator to determine whether the purchase amount is at least KSh 10,000.

iv) If the customer qualifies, calculate the discount using:

Discount = 15% × Purchase Amount

v) Calculate the final amount using:

Final Amount = Purchase Amount − Discount

vi. Display the purchase amount, discount and final amount.

Task 3: Student Examination Results

A lecturer wants to calculate the final mark for a student using coursework and examination marks. Write a Python program that performs the following tasks:

i. Ask the user to enter:

- Student name
- Coursework mark out of 30
- Examination mark out of 70

ii. Calculate the total mark using:

Total Mark = Coursework Mark + Examination Mark

iii. Use comparison operators to determine whether the student has achieved at least 40 marks.

iv) Display the student's name, coursework mark, examination mark and total mark.

v) Display True or False to indicate whether the student has passed.

4. TOPIC 4: PYTHON DATA STRUCTURES

Task 1: Student Marks Using a List

A lecturer wants to store the marks of a student in five subjects. Write a Python program that performs the following tasks:

i. Create a list named marks containing five subject marks.
ii. Display all the marks.
iii. Calculate and display the total marks using sum().
iv. Calculate and display the average mark.
v. Display the highest mark using max().
vi. Display the lowest mark using min().
vii. Modify the mark of the third subject and display the updated list.

Task 2: Customer Details Using a Tuple

A company wants to store customer information that should not be modified during program execution. Write a Python program that performs the following tasks:

i. Create a tuple named customer containing:

- Customer name
- Customer ID
- Telephone number
- Email address

ii. Display the complete tuple.

iii. Access and display the customer's name and telephone number using their indexes.

iv) Display the number of items in the tuple using len().

v) Attempt to modify the customer's telephone number.

vi. State why the modification cannot be performed on the tuple.

Task 3: Unique Product Categories Using a Set

An online shop receives product categories from different departments. Some categories are entered more than once. Write a Python program that performs the following tasks:

i. Create a set named categories containing at least six product categories, including two duplicate categories.
ii. Display the set.
iii. Add a new product category using add().
iv. Remove one category using remove().
v. Display the final set.
vi. Explain how using a set helps the company maintain unique product categories.

5. TOPIC 5: DECISION MAKING

Task 1: Student Grading System

A university wants to automate student grading.

Write a Python program that performs the following tasks:

i. Ask the student to enter their name and examination mark.

ii. Convert the mark to an integer.

iii. Use an if...elif...else statement to assign grades as follows:

- `70–100 → A`
- `60–69 → B`

- 50–59 → C
- 40–49 → D
- Below 40 → F

iv) If the mark is below 0 or above 100, display:

```
Invalid mark
```

v) Display the student's name, mark and grade.

Task 2: ATM Withdrawal

A bank wants to develop a simple ATM withdrawal program. The customer's account has a balance of KSh 50,000. Write a Python program that performs the following tasks:

i. Store the initial account balance as KSh 50,000.
ii. Ask the customer to enter the withdrawal amount.
iii. Convert the amount to an integer.
iv. Check that:
    - The withdrawal amount is greater than zero.
    - The withdrawal amount does not exceed the account balance.
    - The withdrawal amount is a multiple of KSh 500.
v. If all conditions are satisfied, deduct the amount from the balance.
vi. Display the remaining balance.
vii. Display an appropriate error message when any of the conditions is not satisfied.

Task 3: Employee Bonus

A company awards employees a bonus if they have worked for at least three years and have achieved a performance rating of 4 or 5. Write a Python program that performs the following tasks:

i. Ask the employee to enter:

- Employee name
- Years of service
- Performance rating

ii. Convert the years of service and performance rating to integers.

iii. Use logical operators to check whether:

- Years of service is at least 3 **AND**
- Performance rating is at least 4.

iv) If the employee qualifies, display:

```
Employee qualifies for a bonus
```

v) Otherwise, display:

```
Employee does not qualify for a bonus
```

vi. Display the employee's details and eligibility result.

6. TOPIC 6: PYTHON FUNCTIONS

Task 1: Electricity Bill Function

An electricity company wants to calculate customers' electricity bills. Write a Python program that performs the following tasks:

i. Define a function named *calculate_bill().*
ii. The function should accept:
    - Units consumed
    - Cost per unit
iii. Calculate the bill using:

Bill = Units Consumed × Cost Per Unit

iv. Return the calculated bill.
v. Ask the user to enter the units consumed and cost per unit.
vi. Call the calculate_bill() function.
vii. Display the calculated electricity bill.

Task 2: Student Grade Function

A university wants to reuse the same grading procedure for different students.

Write a Python program that performs the following tasks:

i. Define a function named calculate_grade(mark).
ii. The function should accept the student's mark as a parameter.
iii. Use the following grading criteria:

```
70–100 → A
60–69 → B
```

```
50–59 → C
40–49 → D
Below 40 → F
```

iv. Return the appropriate grade.
v. Ask the user to enter a student's mark.
vi. Call the function and display the returned grade.

Task 3: Temperature Conversion Function

A weather application needs to convert temperature from Celsius to Fahrenheit. Write a Python program that performs the following tasks:

i. Define a function named convert_temperature().
ii. Accept the Celsius temperature as a parameter.
iii. Convert Celsius to Fahrenheit using:

Fahrenheit = (Celsius × 9/5) + 32

iv. Return the Fahrenheit value.
v. Ask the user to enter the temperature in Celsius.
vi. Call the function and display the converted temperature to two decimal places.

7. TOPIC 7: PYTHON DICTIONARIES

Task 1: Student Record

A university wants to store student information using a dictionary.

Write a Python program that performs the following tasks:

i. Create a dictionary named student containing:

- Registration number
- Student name
- Course
- Year of study

ii. Display the student's name and course using their dictionary keys.

iii. Change the student's year of study.

iv) Add an email address to the dictionary.

v) Display the complete updated dictionary.

Task 2: Inventory Management

A supermarket wants to use a dictionary to manage the quantity of products in stock. Write a Python program that performs the following tasks:

i. Create a dictionary named inventory containing at least five products and their quantities.
ii. Display the quantity of a specified product.
iii. Add a new product to the dictionary.
iv. Update the quantity of an existing product.
v. Remove one product from the dictionary.
vi. Display the final inventory.

Task 3: Employee Salary Records

A company stores employee names and salaries using a Python dictionary. Write a Python program that performs the following tasks:

i. Create a dictionary named employees containing at least five employees and their salaries.
ii. Display all employee names and salaries using a loop.
iii. Update the salary of one employee.
iv. Add a new employee and salary.
v. Calculate and display the total salary payable to all employees.

8. TOPIC 8: PYTHON FILE HANDLING

Task 1: Student Records File

A lecturer wants to save student information in a text file. Write a Python program that performs the following tasks:

i. Open a file named students.txt in **write mode (w)**.
ii. Write the following information for at least three students:

- Student name
- Registration number
- Course
- Mark

iii. Use print(..., file=f) or an equivalent method to write the information to the file.
iv. Close the file using close().
v. State what would happen to the existing contents of the file if it already exists and is opened using w.

Task 2: Sales Records

A supermarket stores daily sales in a file named sales.txt. Write a Python program that performs the following tasks:

i. Open sales.txt in **append mode (a)**.
ii. Ask the user to enter:
    - Product name
    - Quantity sold
    - Total sales amount
iii. Write the new sales record to the file.
iv. Close the file.
v. Explain why append mode is appropriate for this application.

Task 24: Reading Customer Records

A company has stored customer information in a file called customers.txt. Write a Python program that performs the following tasks:

i. Open customers.txt in **read mode (r)**.
ii. Read the contents of the file.
iii. Display the contents on the screen.
iv. Close the file.
v. State the purpose of each of the following file modes:
    - r
    - w
    - a

9. TOPIC 9: EXCEPTION HANDLING

Task 1: ATM Input Validation

An ATM requires customers to enter the amount they want to withdraw. A customer may accidentally enter text instead of a number. Write a Python program that performs the following tasks:

i. Ask the customer to enter the withdrawal amount.
ii. Convert the entered value to an integer.
iii. Place the conversion operation inside a try block.
iv. Use an except block to handle ValueError.
v. Display:

```
Invalid amount. Please enter a number.
```

when invalid data is entered.

vi. Display the withdrawal amount when a valid number is entered.

Task 2: Student Age Registration

A university registration system requires students to enter their age. Write a Python program that performs the following tasks:

i. Ask the student to enter their age.
ii. Convert the input to an integer.
iii. Use ***try...except*** to handle invalid input.
iv. If the user enters a valid number, display:

```
Age successfully recorded
```

v. If the user enters text such as "twenty", display:

```
Invalid age entered
```

vi. Ensure that the program does not terminate unexpectedly because of the invalid input.

Task 3: File Exception Handling

A company has a program that reads employee information from employees.txt. The file may not exist when the program is executed. Write a Python program that performs the following tasks:

i. Use a try block to open *employees.txt* in read mode.
ii. Read and display the contents of the file.
iii. Use an ***except FileNotFoundError*** block to display:

```
    The employee file was not found.
```

iv. Use a finally block to display:

```
    File processing completed.
```

v. Ensure that the program handles the missing file without terminating unexpectedly.

10. TOPIC 10: CLASSES AND OBJECTS

Task 1: Bank Account

A bank wants to develop a Python program to manage customer bank accounts.

Write a Python program that performs the following tasks:

i. Create a class named BankAccount with the following data members:
- Account number
- Customer name
- Balance

ii. Include the following public methods:
- deposit() → to add money to the account balance
- withdraw() → to deduct money from the account balance
- displayAccount() → to display account information and current balance

iii. In the main section:
- Create an object of the BankAccount class.
- Assign account details.
- Deposit KSh 5,000.
- Withdraw KSh 2,000.
- Display the account details and updated balance.

Task 3: Student Management

A university wants to represent students as objects. Write a Python program that performs the following tasks:

i. Create a class named Student with the following attributes:

- Student name
- Registration number

- Course
- Mark

ii. Include the following methods:

- setStudentDetails() → to assign student information.
- calculateGrade() → to calculate the grade using:
  - 70–100 → A
  - 60–69 → B
  - 50–59 → C
  - 40–49 → D
  - Below 40 → F
- displayStudentDetails() → to display the student's information and grade.

iii. In the main section:

- Create a Student object.
- Set the student's details.
- Calculate the grade.
- Display the student details and grade.

Task 3: Product Management

An online shop wants to manage products using Python classes. Write a Python program that performs the following tasks:

i. Create a class named Product with:

- Product name
- Product code
- Price
- Quantity

ii. Include the following methods:

- setProductDetails() → to assign product information.
- calculateStockValue() → to calculate:

  Stock Value = Price × Quantity
- displayProductDetails() → to display product information and stock value.

iii. In the main section:

- Create a Product object.
- Set the product details.
- Calculate the stock value.

- Display the product details.

11. TOPIC 11: INHERITANCE

Task 1: University Employees

A university has different categories of employees. All employees have a name and employee ID, while lecturers additionally have a teaching subject.

Write a Python program that performs the following tasks:

i. Create a parent class named Employee with:

- Employee name
- Employee ID

ii. Include a method named displayEmployee() to display the employee details.

iii. Create a child class named Lecturer that inherits from Employee.

iv) Add the following attribute to the Lecturer class:

- Teaching subject

v) Include a method named displayLecturer() to display the employee name, employee ID and teaching subject.

vi. In the main section:

- Create a Lecturer object.
- Assign the employee details.
- Assign the teaching subject.
- Display all information.

Task 2: Bank Accounts

A bank has different types of accounts. All accounts have an account number and balance, while a savings account also has an interest rate. Write a Python program that performs the following tasks:

i. Create a parent class named Account with:

- Account number
- Balance

ii. Include a method named displayAccount().

iii. Create a child class named SavingsAccount that inherits from Account.

iv) Add ***Interest rate*** to the SavingsAccount class.

v) Include a method named calculateInterest() using:

Interest = Balance × Interest Rate / 100

vi. Create a SavingsAccount object and display the account details and calculated interest.

Task 3: Transport System

A transport company operates different types of vehicles.

Write a Python program that performs the following tasks:

i. Create a parent class named Vehicle with:

- Registration number
- Model

ii. Include a method named displayVehicle().

iii. Create a child class named Bus that inherits from Vehicle.

iv) Add:

- Passenger capacity

to the Bus class.

v) Include a method named displayBus() to display all vehicle information.

vi. Create a Bus object and display its details.

12. TOPIC 12: ABSTRACTION AND POLYMORPHISM

Task 1: Payment System

An online shopping system supports different payment methods. Write a Python program that performs the following tasks:

i. Create an abstract class named Payment.

ii. Define an abstract method named makePayment().

iii. Create a class named MobileMoneyPayment that inherits from Payment.

iv) Implement makePayment() in MobileMoneyPayment to display:

```
Payment made using Mobile Money
```

v) Create another class named CardPayment that inherits from Payment.

vi. Implement makePayment() in CardPayment to display:

```
Payment made using Bank Card
```

vii. Create objects of both classes and call makePayment().

viii. Explain how the program demonstrates **abstraction and polymorphism**.

Task 2: Transport Fare System

A transport company calculates fares differently for different types of vehicles. Write a Python program that performs the following tasks:

i. Create a parent class named Vehicle.

ii. Define a method named calculateFare().

iii. Create a class named Bus that implements calculateFare() using:

Fare = Distance × 50

iv) Create a class named Taxi that implements calculateFare() using:

Fare = Distance × 100

v) Create one Bus object and one Taxi object.

vi. Pass a distance of 10 kilometres to both objects.

vii. Display the calculated fare for each vehicle.

viii. Explain how calling the same calculateFare() method produces different results for the two objects.

Task 3: Shape Area Calculator

An engineering application needs to calculate the area of different shapes. Write a Python program that performs the following tasks:

i. Create an abstract class named Shape.

ii. Define an abstract method named area().

iii. Create a class named Circle that inherits from Shape.

iv) Implement area() in Circle using:

Area = π × radius²

v) Create a class named Rectangle that inherits from Shape.

vi. Implement area() in Rectangle using:

Area = Length × Width

vii. Create one circle object and one rectangle object.

viii. Calculate and display the area of both shapes.

ix) Explain how the program demonstrates polymorphism.

13. TOPIC 13: INTEGRATED PYTHON PROGRAMMING

Task 1: Student Management System

A university wants to develop a simple Python-based student management system.

Write a Python program that performs the following tasks:

i. Create a class named Student with:

- Student name
- Registration number
- Course
- Mark

ii. Include methods to:

- Set student details.
- Calculate the student's grade.
- Display student details and grade.

iii. Store at least **three student objects in a list**.

iv) Use a function named displayStudents() to display all students.

v) Use try...except to handle invalid marks entered by the user.

vi. Save the student details and grades to a file named students.txt.

vii. Close the file after writing.

Task 2: Supermarket Inventory System

A supermarket wants to automate its inventory management using Python. Write a Python program that performs the following tasks:

i. Create a class named Product with:

- Product name
- Product code
- Price
- Quantity

ii. Include methods to:

- Set product details.
- Calculate stock value using:

  Stock Value = Price × Quantity
- Display product information.

iii. Create at least **three product objects**.

iv) Store the objects in a list.

v) Calculate and display the total value of all products in stock.

vi. Use try...except to handle invalid price or quantity input.

vii. Save the inventory details to inventory.txt.

viii. Close the file after writing.

Task 2: Simple Banking System

A bank wants to develop a basic Python application for managing customer accounts. Write a Python program that performs the following tasks:

i. Create a class named BankAccount with:

- Account number
- Customer name
- Balance

ii. Include methods to:

- Deposit money.
- Withdraw money.
- Display account details.

iii. Ensure that a customer cannot withdraw more money than the available balance.

iv) Use a function named performTransaction() to process a deposit or withdrawal.

v) Use try...except to handle non-numeric transaction amounts.

vi. Create at least **two bank account objects**.

vii. Display the updated balance for each account.

viii. Save the account details and updated balances to a file named accounts.txt.